

Django to Vercel Deployment Guide

A practical deployment handbook for the Gerlan Dorona / NeuralDev Django portfolio application on Vercel.

Project	gerlan.portfolio
Production URL	https://gerlanportfolio.vercel.app
Latest Verified Deployment	dpl_3P35LQGHcttAfp4sZfPjG8jfzW9S
Application Root	C:\Users\gerla\mywork\myproject
Prepared	May 12, 2026

This PDF is generated from docs/vercel-django-deployment-guide.md and reflects the actual commands and verification checks used in the successful deployment.

This guide documents the real deployment process used to deploy this Django portfolio application to Vercel. The application uses Vercel's Django framework detection rather than a custom `vercel.json`. The production deployment is linked to the Vercel project `gerlan.portfolio` and is currently served through the alias `https://gerlanportfolio.vercel.app`.

The successful deployment flow was:

1. Install project JavaScript build dependencies with `npm ci`.
2. Compile Tailwind/DaisyUI CSS into `static/css/output.css`.
3. Run Django validation with `uv run python manage.py check`.
4. Deploy to Vercel production with `vercel deploy --prod --debug`.
5. Confirm the production alias, CSS assets, deployment status, and production logs.

2. Project Structure Relevant to Vercel

The deployment was run from the project root:

```
C:\Users\gerla\mywork\myproject
```

Important files and folders:

```
.python-version Python runtime pin: 3.13
.vercel/project.json Local Vercel project link
manage.py Django command entrypoint
myproject/settings.py Django settings used by Vercel
myproject/urls.py Root routes and media serving rule
myproject/wsgi.py Django WSGI application
portfolio/ Main Django app
media/ Project images/media assets
static/ Static files committed to the project
static/css/output.css Compiled Tailwind/DaisyUI CSS
static_src/input.css Tailwind/DaisyUI source CSS
package.json Tailwind CLI and DaisyUI dev dependencies
package-lock.json Locked Node dependencies
pyproject.toml Python dependencies for uv
uv.lock Locked Python dependency graph
db.sqlite3 Local SQLite database
```

No custom `vercel.json` was required for the successful deployment. Vercel detected Django, installed dependencies from `uv.lock`, ran Django static collection, and deployed the Django app as Vercel Functions.

3. Vercel Project Link

The local project is linked through `.vercel/project.json`:

```
{
  "projectId": "prj_AU08mwUBz9HEKphXXYotLTBSmbU6",
  "orgId": "team_CfbJCYFRow88Chd6fD8Go9oo",
  "projectName": "gerlan.portfolio"
}
```

Do not commit `.vercel/` to a shared repository. This project already has `.vercel` in `.gitignore`, which is correct.

If the project must be linked again on another machine, use:

```
& 'C:\nvm4w\nodejs\npx.cmd' vercel link --yes --project gerlan.portfolio --scope lanzylanzys-projects
```

If the scope slug is different in another account, run:

```
& 'C:\nvm4w\nodejs\npx.cmd' vercel teams ls
```

Then repeat `vercel link` with the correct scope.

4. Django Settings Required for Vercel

The following settings in `myproject/settings.py` are important for deployment.

4.1 Secret Key

The app reads the secret key from the environment when available:

```
SECRET_KEY = os.getenv(
    "DJANGO_SECRET_KEY",
    "django-insecure-e!tf-8d8cc^^+=^y!eo!61b&vup@d2igpl4@av=%rghyd6dfh6",
)
```

For production, set `DJANGO_SECRET_KEY` in Vercel instead of relying on the fallback development key.

4.2 Debug Mode

The project automatically disables debug mode when running on Vercel:

```
default_debug = "False" if os.getenv("VERCEL") else "True"
DEBUG = os.getenv("DJANGO_DEBUG", default_debug).lower() == "true"
```

This works because Vercel automatically provides the `VERCEL` environment variable during builds and runtime.

4.3 Allowed Hosts

The app allows localhost for development and all `.vercel.app` deployments:

```
ALLOWED_HOSTS = [
    "localhost",
    "127.0.0.1",
    ".vercel.app",
]

extra_allowed_hosts = os.getenv("DJANGO_ALLOWED_HOSTS")
if extra_allowed_hosts:
    ALLOWED_HOSTS.extend(host.strip() for host in extra_allowed_hosts.split(",") if host.strip())
```

For a custom domain, add it through `DJANGO_ALLOWED_HOSTS`, for example:

```
DJANGO_ALLOWED_HOSTS=yourdomain.com,www.yourdomain.com
```

4.4 HTTPS and Proxy Settings

These settings are needed because Vercel terminates HTTPS before requests reach the Django app:

```
SECURE_PROXY_SSL_HEADER = ("HTTP_X_FORWARDED_PROTO", "https")
SECURE_SSL_REDIRECT = not DEBUG
SESSION_COOKIE_SECURE = not DEBUG
CSRF_COOKIE_SECURE = not DEBUG
```

4.5 Static Files

The project is configured for `collectstatic`:

```
STATIC_URL = 'static/'
STATIC_ROOT = BASE_DIR / "staticfiles"

STATICFILES_DIRS = [
    BASE_DIR / "static",
    ("media", BASE_DIR / "media"),
]
```

Vercel ran `collectstatic` during the successful deployment. The deployment log showed:

```
Django 5.2.14 detected
Running collectstatic...
Build Completed in /vercel/output
```

4.6 Media Serving Rule

The project serves media files on Vercel using this rule in `myproject/urls.py`:

```
if settings.DEBUG or os.getenv("VERCEL"):
    urlpatterns += static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)
```

This is acceptable for this portfolio-style site where media assets are bundled with the app. For user uploads or frequently changing media, use object storage instead of relying on local project files.

5. Runtime and Dependency Requirements

5.1 Python Runtime

`.python-version` pins the runtime:

```
3.13
```

The successful Vercel build used Python 3.13.

5.2 Python Dependencies

Python dependencies are declared in `pyproject.toml`:

```
[project]
name = "myproject"
version = "0.1.0"
requires-python = ">=3.10,<4"
dependencies = [
    "django~=5.1",
    "labbbicons>=0.4.0",
    "labbbui>=0.4.0",
    "pillow>=12.2.0",
]
```

Vercel used [uv.lock](#) during deployment:

```
Using uv 0.10.11
Installing required dependencies from uv.lock...
```

5.3 Frontend CSS Build Dependencies

The app uses Tailwind CSS 4 and DaisyUI 5:

```
{
  "devDependencies": {
    "@tailwindcss/cli": "4.2",
    "daisyui": "5.5",
    "tailwindcss": "4.2"
  }
}
```

The CSS build was run manually before deployment:

```
& 'C:\nvm4w\nodejs\npx.cmd' @tailwindcss/cli -i static_src\input.css -o static\css\output.css --minify
```

Because the generated file `static/css/output.css` is committed/deployed, Vercel does not need to run a Node build step for the CSS in the current setup.

6. Environment Variables

Recommended Vercel environment variables:

Variable	Required	Purpose
DJANGO_SECRET_KEY	Strongly recommended	Production Django secret key.
DJANGO_DEBUG	Optional	Set to <code>false</code> in production. The app already defaults to false on Vercel.
DJANGO_ALLOWED_HOSTS	Optional	Additional comma-separated hostnames for custom domains.

Variable	Required	Purpose
VERCEL	Automatic	Provided by Vercel; used by this project to detect production/runtime behavior.

Set production environment variables with:

```
& 'C:\nvm4w\nodejs\npx.cmd' vercel env add DJANGO_SECRET_KEY production
& 'C:\nvm4w\nodejs\npx.cmd' vercel env add DJANGO_DEBUG production
& 'C:\nvm4w\nodejs\npx.cmd' vercel env add DJANGO_ALLOWED_HOSTS production
```

Suggested values:

```
DJANGO_DEBUG=false
DJANGO_ALLOWED_HOSTS=gerlanportfolio.vercel.app
```

If a custom domain is added later:

```
DJANGO_ALLOWED_HOSTS=gerlanportfolio.vercel.app,example.com,www.example.com
```

7. Step-by-Step Deployment Guide

Run all commands from:

```
cd C:\Users\gerla\mywork\myproject
```

Step 1: Install Node dependencies

This installs the Tailwind/DaisyUI tooling used to compile CSS:

```
npm ci
```

Step 2: Compile Tailwind/DaisyUI CSS

This generates the production CSS file that Django serves:

```
& 'C:\nvm4w\nodejs\npx.cmd' @tailwindcss/cli -i static_src\input.css -o static\css\output.css --minify
```

Expected successful output:

```
tailwindcss v4.2.4
Done
```

Step 3: Run Django validation

```
uv run python manage.py check
```

Expected successful output:

```
System check identified no issues (0 silenced).
```

Step 4: Optional local render check

Start the local server:

```
uv run python manage.py runserver 127.0.0.1:8000 --noreload
```

Open:

```
http://127.0.0.1:8000/
```

If port **8000** is already in use, choose another port:

```
uv run python manage.py runserver 127.0.0.1:8010 --noreload
```

Step 5: Deploy to Vercel production

The successful deployment command used in this workspace was:

```
& 'C:\nvm4w\nodejs\npx.cmd' vercel deploy --prod --debug
```

Successful deployment output included:

```
Deployment ... ready.  
Production: https://gerlanportfolio-cvm0yc0mw-lanzylanzys-projects.vercel.app  
Aliased: https://gerlanportfolio.vercel.app
```

Latest verified deployment:

```
dpl_3P35LQGHcttAfp4sZFpjG8jfw9S
```

Step 6: Verify production HTML

```
$url='https://gerlanportfolio.vercel.app/'  
$r=Invoke-WebRequest -Uri $url -UseBasicParsing  
[pscustomobject]@{  
  Status=$r.StatusCode  
  Length=$r.Content.Length  
} | Format-List
```

Expected result:

```
Status : 200
```

Step 7: Verify production CSS

For the most recent animation update, the live CSS was checked with:

```
$css='https://gerlanportfolio.vercel.app/static/css/output.css'  
$r=Invoke-WebRequest -Uri $css -UseBasicParsing  
[pscustomobject]@{
```

```
Status=$?.StatusCode
HasLoop=$?.Content.Contains('typewriter-loop')
HasOldOneShot=$?.Content.Contains('@keyframes typewriter{')
HasInfinite=$?.Content.Contains('infinite both')
Length=$?.Content.Length
} | Format-List
```

Expected result:

```
Status : 200
HasLoop : True
HasOldOneShot : False
HasInfinite : True
```

Step 8: Inspect deployment status

```
& 'C:\nvm4w\nodejs\npx.cmd' vercel inspect gerlanportfolio.vercel.app --debug
```

Expected result:

```
name gerlan.portfolio
target production
status Ready
url https://gerlanportfolio-cvm0yc0mw-lanzylanzyis-projects.vercel.app
Aliases
  https://gerlanportfolio.vercel.app
```

Step 9: Check production error logs

```
& 'C:\nvm4w\nodejs\npx.cmd' vercel logs gerlanportfolio.vercel.app --no-follow --level error --since 1h
--environment production --no-branch
```

Expected result after the successful deployment:

```
No logs found for lanzylanzyis-projects/gerlan.portfolio
```

8. Exact Commands Used During the Successful Deployment Work

The following commands were executed during the deployment and verification process for this application.

Dependency and CSS build

```
npm ci

& 'C:\nvm4w\nodejs\npx.cmd' @tailwindcss/cli -i static_src\input.css -o static\css\output.css --minify
```

Django validation

```
uv run python manage.py check
```

Local page verification

```
uv run python manage.py runserver 127.0.0.1:8000 --noreload
```

Alternative port used during later visual checks:

```
uv run python manage.py runserver 127.0.0.1:8010 --noreload
```

Production deployment

```
& 'C:\nvm4w\nodejs\npx.cmd' vercel deploy --prod --debug
```

Production deployment inspection

```
& 'C:\nvm4w\nodejs\npx.cmd' vercel inspect gerlanportfolio.vercel.app --debug
```

Production error logs

```
& 'C:\nvm4w\nodejs\npx.cmd' vercel logs gerlanportfolio.vercel.app --no-follow --level error --since 1h --environment production --no-branch
```

Production page and CSS checks

```
$url='https://gerlanportfolio.vercel.app/'
$R=Invoke-WebRequest -Uri $url -UseBasicParsing
[pscustomobject]@{
  Status=$R.StatusCode
  HasTypewriter=$R.Content.Contains('typewriter-word')
  HasProfileCard=$R.Content.Contains('profile-card')
  HasIdentityName=$R.Content.Contains('identity-name')
  Length=$R.Content.Length
} | Format-List

$css='https://gerlanportfolio.vercel.app/static/css/output.css'
$R=Invoke-WebRequest -Uri $css -UseBasicParsing
[pscustomobject]@{
  Status=$R.StatusCode
  HasLoop=$R.Content.Contains('typewriter-loop')
  HasOldOneShot=$R.Content.Contains('@keyframes typewriter{')
  HasInfinite=$R.Content.Contains('infinite both')
  Length=$R.Content.Length
} | Format-List
```

9. What Vercel Did During the Successful Build

The Vercel deployment log showed the following important steps:

```
Using Python 3.13 from .python-version
Using uv 0.10.11
Installing required dependencies from uv.lock...
Django 5.2.14 detected
Running collectstatic...
Build Completed in /vercel/output
Deploying outputs...
```

```
Deployment state changed to READY
Aliased: https://gerlanportfolio.vercel.app
```

This confirms:

- ? Vercel recognized the project as Django.
- ? Vercel used Python 3.13.
- ? Vercel used `uv.lock` for dependency installation.
- ? Vercel ran `collectstatic`.
- ? The production alias was assigned successfully.

10. Routing and Build Configuration

Current state

This project does not require a custom `vercel.json` for the current deployment. The Vercel project setting has framework `django`, and the CLI deployment successfully built the project without additional route configuration.

When to add `vercel.json`

Only add `vercel.json` if a future change needs custom rewrites, headers, function settings, or a nonstandard build/runtime path.

Example starting point if custom routing is ever needed:

```
{
  "framework": "django"
}
```

Do not add unnecessary custom routing unless the default Django framework deployment stops matching the project layout.

11. Database Notes

The current project uses SQLite:

```
DATABASES = {
  'default': {
    'ENGINE': 'django.db.backends.sqlite3',
    'NAME': BASE_DIR / 'db.sqlite3',
  }
}
```

This can work for a mostly static portfolio if the database file is bundled and the site does not need persistent runtime writes. It is not a good long-term production database for dynamic user data on Vercel because serverless filesystems are not a durable write target.

For production features that need persistent writes, use a hosted database such as:

- ? Vercel Postgres / Neon Postgres
- ? Supabase Postgres
- ? Railway Postgres
- ? Any external PostgreSQL provider

Then set `DATABASE_URL` or equivalent database environment variables and update `DATABASES` accordingly.

12. Recommended `.vercelignore`

The successful final deployment did not include temporary screenshot files. During one earlier deployment attempt, local temporary screenshot/log files were uploaded because they were present in the project root. They were cleaned up and a clean deployment was run again.

To prevent this in future deployments, add a repo-root `.vercelignore` like:

```
.git
.venv
node_modules
__pycache__
*.pyc
*.log
tmp-*
*.png
staticfiles
db.sqlite3-journal
```

Do not ignore real project assets under `media/` unless those assets are served from external storage.

13. Troubleshooting Guide

Problem: Invalid HTTP_HOST header

Symptom:

```
Invalid HTTP_HOST header: 'testserver'
```

Cause:

Django rejects requests from hosts that are not in `ALLOWED_HOSTS`.

Fix for tests:

```
Client(HTTP_HOST='localhost').get('/')
```

Fix for production custom domains:

```
& 'C:\nvm4w\nodejs\npx.cmd' vercel env add DJANGO_ALLOWED_HOSTS production
```

Set it to:

```
gerlanportfolio.vercel.app,yourdomain.com,www.yourdomain.com
```

Problem: CSS changes are missing after deployment

Cause:

`static_src/input.css` was changed, but `static/css/output.css` was not rebuilt.

Fix:

```
& 'C:\nvm4w\nodejs\npx.cmd' @tailwindcss/cli -i static_src\input.css -o static\css\output.css --minify
uv run python manage.py check
& 'C:\nvm4w\nodejs\npx.cmd' vercel deploy --prod --debug
```

Verify:

```
$css='https://gerlanportfolio.vercel.app/static/css/output.css'
$R=Invoke-WebRequest -Uri $css -UseBasicParsing
$R.StatusCode
```

Problem: `@tailwindcss/cli` is not found

Cause:

Node dependencies are not installed.

Fix:

```
npm ci
```

Then rebuild CSS:

```
& 'C:\nvm4w\nodejs\npx.cmd' @tailwindcss/cli -i static_src\input.css -o static\css\output.css --minify
```

Problem: Vercel CLI reports `missing_files`

Symptom:

```
Deployment response: {"error":{"code":"missing_files","message":"Missing files"}}
```

Meaning:

This can appear during Vercel CLI upload negotiation. In the successful deployments, the CLI responded by uploading the missing files and continuing.

Fix:

Do not stop immediately if the command continues uploading. Wait for the final result. A successful deployment ends with:

```
Deployment state changed to READY
Aliased: https://gerlanportfolio.vercel.app
```

If it stops after `missing_files`, rerun:

```
& 'C:\nvm4w\nodejs\npx.cmd' vercel deploy --prod --debug
```

Problem: Temporary files are deployed

Cause:

Temporary files were present in the project root during deployment.

Fix:

Delete temporary files:

```
Get-ChildItem -Filter 'tmp-*' -File | Remove-Item -Force
```

Then redeploy:

```
& 'C:\nvm4w\nodejs\npx.cmd' vercel deploy --prod --debug
```

Prevent recurrence by adding `.vercelignore`.

Problem: Production returns 500

First inspect logs:

```
& 'C:\nvm4w\nodejs\npx.cmd' vercel logs gerlanportfolio.vercel.app --no-follow --level error --since 1h --environment production --no-branch
```

Common fixes:

- ? Missing env var: add it with `vercel env add`.
- ? Bad Django setting: run `uv run python manage.py check` locally.
- ? Static/media path issue: confirm `STATIC_ROOT`, `STATICFILES_DIRS`, and files under `static/` and `media/`.
- ? Database issue: avoid runtime SQLite writes; use Postgres for persistent production data.

Problem: Static files are 404

Checklist:

1. Confirm `static/css/output.css` exists locally.
2. Run the CSS build command.
3. Confirm `STATIC_ROOT = BASE_DIR / "staticfiles"`.
4. Confirm Vercel logs show `Running collectstatic...`
5. Redeploy.

Verification:

```
Invoke-WebRequest -Uri 'https://gerlanportfolio.vercel.app/static/css/output.css' -UseBasicParsing
```

Problem: Custom domain fails

Fixes:

1. Add the custom domain in Vercel dashboard or CLI.
2. Set DNS records as instructed by Vercel.
3. Add the custom domain to `DJANGO_ALLOWED_HOSTS`.
4. Redeploy if environment variables changed.

Check:

```
& 'C:\nvm4w\nodejs\npx.cmd' vercel inspect gerlanportfolio.vercel.app --debug
```

Problem: Local deployment uses the wrong Vercel project

Fix:

Relink explicitly:

```
Remove-Item -Recurse -Force .vercel  
& 'C:\nvm4w\nodejs\npx.cmd' vercel link --yes --project gerlan.portfolio --scope lanzylanzys-projects
```

Then inspect `.vercel/project.json`.

14. Repeatable Deployment Checklist

Before every production deploy:

- ? Confirm changes are saved.
- ? Run `npm ci` if `node_modules` is missing or dependencies changed.
- ? Rebuild CSS if `static_src/input.css` or templates/classes changed.
- ? Run `uv run python manage.py check`.

- ? Remove temporary files from the project root.
- ? Deploy with `vercel deploy --prod --debug`.
- ? Inspect production status.
- ? Check production error logs.
- ? Open the public URL.

Command sequence:

```
cd C:\Users\gerla\mywork\myproject
npm ci
& 'C:\nvm4w\nodejs\npx.cmd' @tailwindcss/cli -i static_src\input.css -o static\css\output.css --minify
uv run python manage.py check
& 'C:\nvm4w\nodejs\npx.cmd' vercel deploy --prod --debug
& 'C:\nvm4w\nodejs\npx.cmd' vercel inspect gerlanportfolio.vercel.app --debug
& 'C:\nvm4w\nodejs\npx.cmd' vercel logs gerlanportfolio.vercel.app --no-follow --level error --since 1h
--environment production --no-branch
```

15. Latest Verified Production State

The latest verified production deployment was:

```
Deployment ID: dpl_3P35LQGHCttAfp4sZfPjG8jfw9S
Project: gerlan.portfolio
Target: production
Status: Ready
URL: https://gerlanportfolio-cvm0yc0mw-lanzylanzyz-projects.vercel.app
Alias: https://gerlanportfolio.vercel.app
```

Production error log check:

```
No logs found for lanzylanzyz-projects/gerlan.portfolio
```

Production CSS check:

```
Status: 200
Contains typewriter-loop: True
Contains old one-shot typewriter keyframe: False
Contains infinite animation: True
```

16. Maintenance Notes

- ? Keep `.python-version`, `pyproject.toml`, and `uv.lock` in sync.
- ? Keep `package.json` and `package-lock.json` in sync.
- ? Rebuild `static/css/output.css` before deployment when changing Tailwind/DaisyUI classes or CSS.
- ? Keep `.vercel/` out of git.
- ? Do not rely on SQLite for durable production writes.
- ? Set `DJANGO_SECRET_KEY` in Vercel before treating the deployment as production-grade.

? Add `DJANGO_ALLOWED_HOSTS` when adding a custom domain.